

NFHS-5 Health Risk Surveillance

Project Documentation Report

Field	Details
Team	Team 7 — NumPies
Institution	SRM Institute of Science and Technology
Program	Samsung Innovation Campus
Dataset	NFHS-5 (National Family Health Survey), 2019-21
Language	Python 3 FastAPI Pandas NumPy Folium
Repository	github.com/groovycatblaze/samsung-health-surveillance

1. Problem Statement

India is one of the most populous nations in the world, yet its public health data often gets locked inside dense survey PDFs that are hard to compare or act on. The National Family Health Survey (NFHS-5) collected detailed state-level health data across 36 states and union territories between 2019 and 2021, covering everything from childhood anemia to adult obesity and hypertension. The problem is simple: this data exists, but no easy way to rank, cluster, or visually explore it does.

Health disparities across Indian states are massive. Bihar and Jharkhand report anaemia rates in children exceeding 67%, while Kerala sits well below 40%. Without a structured way to compare and group these states, policymakers and researchers have no quick handle on where to direct resources first. There is also the added challenge of indicator interaction, a state might rank low on obesity but high on anemia, making single-metric analysis misleading.

This project addresses that gap by building a full data pipeline that cleans, normalises, scores, and clusters Indian states by health risk and serves it all through an interactive web dashboard.

2. Objectives

- Clean and preprocess NFHS-5 survey data across six key health indicators
- Compute a Composite Health Risk Index (CHRI) using weighted normalisation
- Apply graph-based Breadth-First Search (BFS) to identify clusters of states with similar health profiles
- Expose processed data through a RESTful FastAPI backend with structured JSON endpoints
- Render an interactive choropleth map of India using Folium and GeoJSON boundaries
- Present findings through a state-level ranking dashboard with risk tier classification

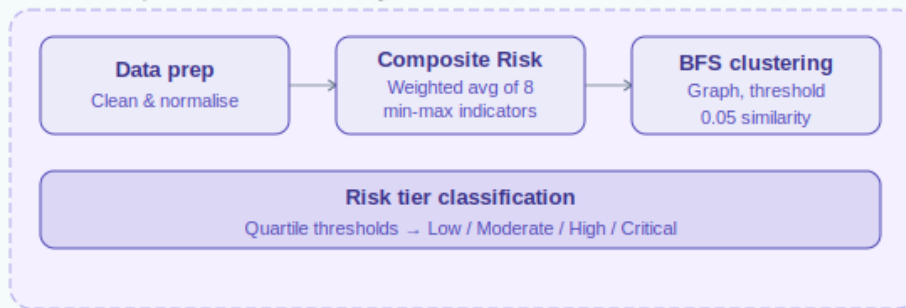
NFHS-5 Health Surveillance — System Architecture

Samsung Innovation Campus · Team 7 NumPies

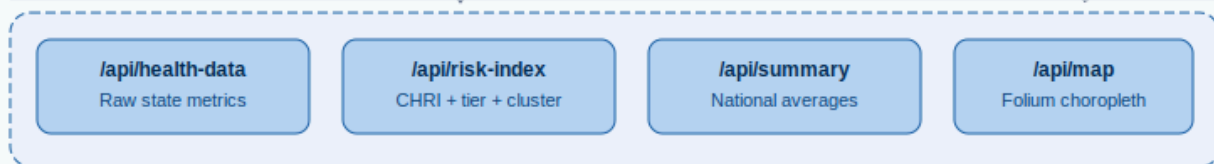
DATA SOURCES



ANALYSIS PIPELINE



FASTAPI BACKEND



FRONTEND DASHBOARD



Legend: Data (Green), Pipeline (Purple), API endpoint (Blue), Frontend component (Orange)

3. Key Code Snippets

3.1 Composite Health Risk Index

Each of the eight health metrics is min-max normalised to $[0, 1]$, then combined using domain-informed weights. Anaemia and obesity carry the highest weights given their prevalence and downstream impact.

```
risk_columns = [
    "Obesity_Women", "Obesity_Men",
    "Anaemia_Children", "Anaemia_Women",
    "BloodSugar_Women", "BloodSugar_Men",
    "Hypertension_Women", "Hypertension_Men",
]

for col in risk_columns:
```

```

mn, mx = risk_df[col].min(), risk_df[col].max()
risk_df[col] = (risk_df[col] - mn) / (mx - mn)

risk_df["Composite_Risk"] = (
    0.20 * risk_df["Obesity_Women"]
+ 0.15 * risk_df["Obesity_Men"]
+ 0.20 * risk_df["Anaemia_Children"]
+ 0.15 * risk_df["Anaemia_Women"]
+ 0.10 * risk_df["BloodSugar_Women"]
+ 0.10 * risk_df["BloodSugar_Men"]
+ 0.05 * risk_df["Hypertension_Women"]
+ 0.05 * risk_df["Hypertension_Men"]
)

```

3.2 BFS State Clustering

States are treated as nodes in a graph. Two states are connected with an edge if their composite risk scores differ by less than 0.05 (the similarity threshold). BFS then traverses this graph to identify connected components, i.e. clusters of states that share similar overall health profiles.

```

THRESHOLD = 0.05
graph = {s: [] for s in states}

for i in range(len(states)):
    for j in range(i + 1, len(states)):
        if abs(scores[i] - scores[j]) <= THRESHOLD:
            graph[states[i]].append(states[j])
            graph[states[j]].append(states[i])

visited, clusters = set(), []
for state in graph:
    if state not in visited:
        queue = deque([state])
        visited.add(state)
        cluster = []
        while queue:
            node = queue.popleft()
            cluster.append(node)
            for nb in graph[node]:
                if nb not in visited:
                    visited.add(nb)
                    queue.append(nb)
        clusters.append(cluster)

```

3.3 FastAPI Endpoint Risk Index

The `/api/risk-index` endpoint returns each state's computed score, tier classification, BFS cluster ID, and geographic metadata as a JSON array, ready to be consumed by any frontend.

```
@app.get("/api/risk-index")
def get_risk_index():
    result = []
    for _, row in _risk_df.iterrows():
        abbr, region = STATE_META.get(row["State"], ("??",
"Other"))
        result.append({
            "rank"           : int(row["Rank"]),
            "state"          : row["State"],
            "abbr"           : abbr,
            "region"         : region,
            "composite_risk":
round(float(row["Composite_Risk"]), 4),
            "risk_tier"      : row["Risk_Tier"],
            "bfs_cluster"   : int(row["BFS_Cluster"]),
        })
    return JSONResponse(content=result)
```

4. Input

The primary data source is the NFHS-5 dataset (2019-21), which is either loaded from a local CSV at `data/nfhs5.csv` or served from a hardcoded cleaned dictionary embedded directly in the application. The following indicators are used per state:

Indicator	Unit	Weight in CHRI
Obesity (Women)	% of women aged 15-49	20%
Obesity (Men)	% of men aged 15-49	15%
Anaemia (Children under 5)	% prevalence	20%
Anaemia (Women)	% of women aged 15-49	15%
High Blood Sugar (Women)	% prevalence	10%
High Blood Sugar (Men)	% prevalence	10%
Hypertension (Women)	% prevalence	5%
Hypertension (Men)	% prevalence	5%

Additionally, a GeoJSON file of Indian state boundaries (`india_state.geojson`) is required in the `data/` directory to render the Folium choropleth map.

5. Output

The project produces two categories of output: structured API responses and a visual dashboard.

5.1 API Endpoints

- GET /api/health-data — returns raw state-wise metrics with region labels
- GET /api/risk-index — returns CHRI scores, rank, risk tier, and BFS cluster per state
- GET /api/clusters — returns all BFS clusters as grouped state lists
- GET /api/summary — returns national averages and tier distribution counts
- GET /api/map — returns a rendered Folium choropleth as an embeddable HTML fragment

5.2 Risk Tier Classification

States are bucketed into four tiers using quartile thresholds on the CHRI: Low (Q1), Moderate (Q1-Q2), High (Q2-Q3), and Critical (above Q3). Key findings from the analysis:

- Punjab and Puducherry consistently score in the Critical tier due to high obesity + hypertension co-occurrence
- Bihar, Jharkhand, and Madhya Pradesh rank poorly on anaemia despite lower obesity rates, reflecting a different type of health burden
- Manipur and Nagaland fall into the Low risk tier, the only two states to do so
- BFS clustering groups most northern states together, and identifies an outlier cluster of high-obesity southern states

5.3 Visual Dashboard

The frontend dashboard renders state cards with colour-coded risk tiers, a ranked list of all 31 states, and an embedded interactive choropleth map. Users can hover over states on the map to see the top two contributing indicators and cluster membership.

6. Conclusion

This project showed that public health data, even at a national scale, can be made genuinely actionable with a relatively lean Python stack. The composite index gave us a single comparable number per state while the BFS clustering revealed natural groupings that do not always follow geographic or economic lines.

The biggest learning for the team was understanding how much preprocessing matters — state names in the NFHS dataset are inconsistent and had to be mapped manually before any merge with GeoJSON worked. We also learned that weighting decisions in composite indices are inherently subjective, so we made sure to document our reasoning and keep the weights easy to adjust in the codebase.

Going forward, the project could be extended to district-level analysis using NFHS-5 district fact sheets, incorporate a time-series layer comparing NFHS-4 vs NFHS-5, and include socioeconomic covariates such as per capita income and literacy rates. The FastAPI architecture was deliberately chosen to make this kind of expansion straightforward.